PROF. DR.-ING. MARK SCHUTERA

FULL STACK HANDWERK

UNFINISHED LECTURE NOTES

Copyright © 2026 Prof. Dr.-Ing. Mark Schutera

PUBLISHED BY UNFINISHED LECTURE NOTES

Combobulated with the help of multiple large language model driven tools. Licensed under the Creative Commons Attribution-NonCommercial 4.0 International License ("CC BY-NC-SA 4.0"). You may not use this file for commercial purposes. If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original You must obtain explicit permission from the author for uses beyond those permitted by this license. To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-sa/4.0/. Unless required by applicable law or agreed to in writing, distributed material is provided on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the license for details.

Last updated, April 2026 — v0.0.1

“SEINE WORT’ UND WERKE MERKT ICH UND DEN BRAUCH,

UND MIT GEISTESSTÄRKE TU ICH WUNDER AUCH.”

DER ZAUBERLEHRLING

Contents

Projektbericht 9

Development and Production 11

Version Everything: Git & GitHub 23

Dependencies & Structure 35

Index 47

Introduction

This is an adaptation of the MIT Missing Semester course material. And oh boy, we need it here in Germany as well! In the Länd of the "Ingenieure", "Handwerk", and "Lehrlings" - we forgot the importance of practice and mastering our tooling skills.

Original course URL https://missing.csail.mit.edu/

As a mechanical engineer by training, I have always had this nagging feeling that something important was missing from my education. The more I transitioned into software engineering roles, the more this feeling intensified. Looking at your curriculum, there is no shortage of advanced concepts and theories to learn. This one fills the gap that many computer science programs overlook: practical proficiency with essential tools and workflows that every software engineer relies on. And truth be told, Professors rely on students having this basic proficiency already acquired, or to figure this stuff out on their own - which is fair in academia. For me, software engineering is also a practical discipline, and ought to be taught as such. I need you to know this stuff, so it is also fair to teach it.

This course will teach you everything you need to know to become proficient with the command line, version control systems like Git, text editors, shell scripting, and other essential tools that are the backbone of modern software development - in the end you will be able to develop and deploy your software solutions. This will make you a better software engineer, and it will make your life easier latest when you need to go full ML Lifecycle.

Projektbericht

Real artists ship. - Steve Jobs

The Why

A lecture that teaches tools and workflows must also assess tools and workflows. Written exams test recall; a project report tests whether you can actually build and deploy something. The Projektbericht is your proof that you have walked the path from empty virtual machines to a running, deployed application.

Pick any block from the course and build a meaningful extension onto it. The extension should add functionality, improve reliability, or strengthen security in a way that goes beyond what the lecture covered. The choice is yours, but each extension must be unique across the class: submit your proposal for approval on a first come, first served basis with me. Once an extension is taken, no other student may choose the same one.

Submit a written report as PDF. Include your name, matriculation number, and a link to your snapshot. The report must cover:

- Which block you chose and what it covers in the course.
- What you built on top of it, which tools and techstack you used, and why it adds value.
- Problems you encountered and how you resolved them.
- What works, what remains incomplete, and what would you do next.

Evaluation considers three dimensions: *depth* of the extension beyond the lecture material, *understanding* demonstrated through explanation and a VM snapshot, and *quality* of the writing.

The Projektbericht replaces a traditional exam. You submit a written report documenting your decisions, your architecture, your modular extension, and the problems you solved along the way.

Nudge replicate all course VM snapshots for Apple Silicon using UTM, so future students on ARM Macs can follow along natively. This means at least 3 Blocks per student, that benefits every cohort after yours.

Looking for inspiration? Explore what tools modern teams actually use at `https://stackshare.io` or browse the ThoughtWorks Technology Radar at `https://www.thoughtworks.com/radar/tools`. Pick something that excites you; the best reports come from genuine curiosity.

Development and Production

Divide and conquer.

The Why

Most software engineering curricula focus on algorithms, data structures, and the theory of computation. Writing a Python script that runs once on a single laptop is a skill; making that script available as a service, keeping it running, and recovering when it fails is a different one. This course addresses the second set of skills, the tools and workflows that sit between working code and the full stack of setting up and maintaining a running system.

Across ten lecture blocks we build PixelWise, a handwritten digit recognition web application. The starting point is two empty virtual machines; the end point is a deployed service including a frontend, and a backend with a trained model, an API, and a database, all running on Linux servers and connected over a network, with version control, automated deployment, persistent storage, and monitoring. Each tool we introduce is motivated by a specific need that arises in the construction of a fullstack application: Pixelwise.

PixelWise is the narrative thread of the entire course. Users draw a digit on a pixelated 28×28 canvas in the browser, the backend runs an image classifier trained on MNIST, the model predicts class and confidence, and results are stored in a database.

This enables you to deploy your own prooducts and services in the future.

Development and Production.

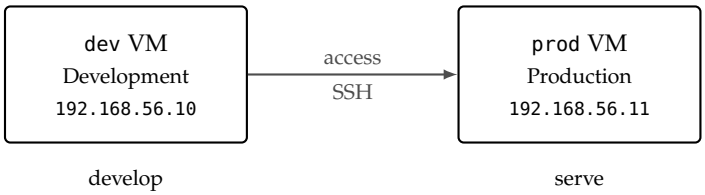

Figure 1: The two-machine setup. The dev VM is where code is written and tested; the prod VM is where the deployed application runs. Code travels from dev to prod through a deliberate deployment step over SSH.

The figure shows the starting arrangement: two virtual machines that play distinct roles throughout the course. The `dev` VM is the development environment in which code is written, modified, and tested. The `prod` VM is the production environment in which the

deployed application runs. Later blocks add the backend stack, the frontend, persistence, and automation between the two, but the separation between development and production stays intact from this block onward.

Hands On Experience

CONSIDER A SMALL WEB APPLICATION running on your laptop. A classmate two time zones away wants to try it; you send them the address. It works, until you close the lid to catch a train. That evening the Wi-Fi has changed, the address is different, and the process you started in the terminal is gone.

A SERVER is a machine whose job is to be available when no one is watching. It stays powered on, holds a stable address, and answers requests around the clock. Your laptop is where code is written and changed; the server is where code meets users. Keeping these two roles on separate machines is the first design decision of the course.

COUNT THE HOURS your laptop is actually reachable at a stable address. Subtract the time it is closed, asleep, on battery, behind a café router, or rebooting after an update. What remains is the service level a laptop alone can offer.

TWO MACHINES, ONE MENTAL MODEL. `dev` at 192.168.56.10 stands in for your laptop. `prod` at 192.168.56.11 stands in for the always-on host. Later lectures fill in the bridge.

THIS FIRST BLOCK focuses on machines and environments rather than on code. Before introducing a programming language, a version control system, or a framework, we need a place which is always up, to run programs, isolated from experiments, and a place for development. By the end of the block you will have provisioned both virtual machines,

- set up a server and a development machine as virtual machines on your laptop,
- learned the subset of Linux needed to navigate a server from the command line,
- and configured remote access over SSH using key-based authentication.

Virtual Machines

A VIRTUAL MACHINE IS A SIMULATED COMPUTER running inside your real computer. The software that creates and runs virtual machines is called a hypervisor. The hypervisor partitions the host's CPU, memory, and disk, and presents each virtual machine with what appears to be its own hardware. From the guest operating system's perspective, it is running on a physical machine.

HYPERVISORS are often categorised as type 1 or type 2. A type 1 hypervisor runs directly on the hardware, with the host operating system built into the hypervisor itself. Examples include VMWare ESXi and Microsoft Hyper-V in its server role; cloud providers use type 1 hypervisors to host the virtual machines you rent. A type 2 hypervisor runs as an application on top of a regular operating system. Oracle VirtualBox Manager is a type 2 hypervisor, which is why we can install it as a normal program on Windows, macOS, or Linux. The conceptual model is the same in both cases; the differences are in performance and deployment context.

ORACLE VIRTUALBOX MANAGER at `https://www.oracle.com/virtualization/virtualbox/` is the hypervisor we install on the host. Virtmanager and VMWare serve the same role with different trade-offs, and we could run this course on any of them.

ISOLATION is the primary reason to use a virtual machine. A misconfiguration, an uninstalled package, or a destructive command inside the guest does not affect the host operating system. Because the hypervisor abstracts away the hardware, the same VM runs identically on Windows, macOS, and Linux hosts, which is the basis of the reproducibility argument.

THERE ARE COSTS. A virtual machine executes a full operating system on top of the host, which carries a performance overhead that is typically small for I/O-bound work and larger for CPU-bound work. Each VM reserves some amount of RAM, disk space, and CPU time from the host, so the number of concurrent VMs is limited by available resources. Boot time is longer than launching an application but shorter than booting physical hardware. These costs are the price paid for isolation, reproducibility, and rollback.

A SNAPSHOT captures the entire machine state at a point in time, including disk contents, memory, and running processes. Snapshots are inexpensive to create and well-suited to exploratory work; making one before a risky change and restoring it afterwards is a standard pattern.

HOST-ONLY NETWORKING puts `dev` and `prod` on a private subnet, typically 192.168.56.0/24, that only the host and the guests can reach. The guests get internet through a second, NAT-mode adapter, so they can install packages without being exposed to the outside world. We assign static IPs, 192.168.56.10 for `dev` and 192.168.56.11 for `prod`, so that every command in the course refers to the same addresses.

OUTSIDE THE CLASSROOM, a production host is usually a rented or purchased server in a data centre, and the development machine is the laptop or workstation in front of you. Running both as VMs inside a single laptop is a teaching shortcut: it preserves the two-machine separation without asking you to rent hardware, and everything we learn here transfers directly when `prod` is later replaced by a cloud instance reached over the public internet.

Linux on the Server

LINUX is a family of Unix-like operating systems built around a common kernel, first released by Linus Torvalds in 1991. A distribution combines the kernel with a package manager, system libraries, and user-space tools into a coherent product. Ubuntu, Debian, Fedora, and Arch are distributions that differ in release cadence and defaults but share most of what matters at the command line.

A SERVER IN THIS SECTION is a computer configured to run services and accept connections over a network, typically without a graphical session. You interact with it through a shell, a program that reads commands and prints their output; the shell we use is Bash, the default on Ubuntu.

FIVE COMMANDS are sufficient for the first session. `ls` lists a directory, with `-la` showing permissions, ownership, and hidden files. `cd` changes directory; `cd ..` moves up one level and `cd ~` returns home. `pwd` prints the current path, `mkdir` creates a directory, and `cat` prints a file to the terminal.

UBUNTU at `https://ubuntu.com/` is the distribution we use on both VMs. Long-term support releases receive five years of security updates, which is why we pick the LTS ISO rather than the latest release.

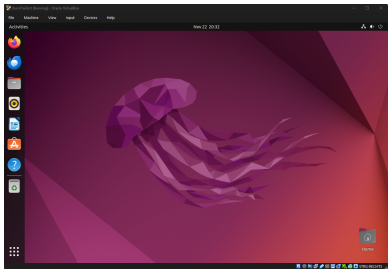

Figure 2: A running Ubuntu guest inside the Oracle VirtualBox Manager. The guest believes it owns a physical machine; the hypervisor is what makes that illusion hold.

LINUX IS A MULTI-USER OPERATING SYSTEM. Every file belongs to a user and a group and carries three permission sets, for owner, group, and everyone else. Each set independently grants read, write, or execute access. `whoami` reports the current user, and `ls -la` shows ownership and permissions:

```
-rw-r--r-- 1 user user 1234 Jan 15 10:00 config.txt
drwxr-xr-x 2 user user 4096 Jan 15 10:00 mydir/
```

THE UNIX PHILOSOPHY shapes the design of these tools. Each program does one thing well, programs pass streams of text to each other, and configuration lives in plain files. The result is a small vocabulary of composable commands that combine into pipelines no single command was designed for.

TWO COMMANDS modify these. `chmod` changes permissions and `chown` changes ownership:

```
chmod 600 ~/.ssh/id_ed25519          # owner-only read and write
sudo chown user:user file.txt        # assign user and group
```

READING THE MODE STRING from left to right, the first character tells you whether it is a file or directory, then three groups of `rwx` for owner, group, others. A dash means the permission is not granted. The leading `d` marks a directory.

The principle of least privilege applies throughout the course: grant each user, process, or service only the access it needs, so that narrow defaults limit the blast radius of a compromise or a mistake.

SSH, the Only Door

SSH, THE SECURE SHELL, is a protocol for operating a remote machine from a terminal over an encrypted channel. In the setup used in this course, SSH is the only means of access to the `prod` VM: there is no graphical login and no console attached. Misconfiguring SSH can lock a user out of the machine, which is why key material and the daemon's configuration are treated carefully in the exercises.

THE BASIC COMMAND takes the form `ssh <username>@<host>`; for example, `ssh produser@192.168.56.11` opens an encrypted connection to the server as the `produser` account. All input and output travel through this channel until the session is closed with `exit` or `Ctrl+D`.

PASSWORD-BASED AUTHENTICATION is vulnerable to guessing, brute-force attacks, and phishing. SSH supports an alternative in which the user presents a cryptographic key pair. The private key is kept on the client and never transmitted; the public key is placed on the server. During authentication the server issues a challenge that only the holder of the private key can answer, proving identity without the key itself ever leaving the client.

GENERATE AN ED25519 KEY PAIR on `dev`:

```
ssh-keygen -t ed25519 -C "you@example.com"
```

The command writes the private key `id_ed25519` and the public key `id_ed25519.pub` to `~/.ssh/`. The private key must have permissions `600`, owner-only read and write, or SSH refuses to use it.

COPY THE PUBLIC KEY TO THE SERVER:

```
ssh-copy-id produser@192.168.56.11
```

This appends the key to `~/.ssh/authorized_keys`. Subsequent connections authenticate using the key pair and do not prompt for a password.

ONCE KEY AUTHENTICATION IS VERIFIED, password authentication can be disabled on the server. Edit `/etc/ssh/sshd_config` to set `PasswordAuthentication no`, then restart the daemon:

```
sudo systemctl restart ssh
```

OPENSSH at `https://www.openssh.com/manual.html` is the implementation shipped with Ubuntu and most other Linux distributions. The client on `dev` and the daemon on `prod` are both part of this project.

PUTTY at `https://www.putty.org/` is a legacy SSH client for Windows, from the years before OpenSSH shipped with the operating system. Windows 10 and later include `ssh.exe` natively, so PuTTY is rarely necessary; you may still encounter it in older tutorials and on locked-down corporate machines.

VS CODE REMOTE-SSH at `https://code.visualstudio.com/docs/remote/ssh` runs the editor locally while the language server, terminal, and file system live on the remote host. It reads `~/.ssh/config` and reuses the same keys as the `ssh` command, so any host you can reach with `ssh user@host` is one click away inside the editor.

ED25519 keys use elliptic-curve cryptography and produce shorter keys than the older RSA algorithm while providing comparable or stronger security. They are the default choice on modern SSH clients.

MANAGING PRIVATE KEYS. Private keys live in `~/.ssh/` on the machine that initiates connections, protected by filesystem permissions and, when generated with a passphrase, by symmetric encryption. `ssh-agent` holds the decrypted key in memory for the session so the passphrase is typed once per login. Password managers such as KeePass at `https://keepass.info/` or 1Password can store the passphrase and, with their SSH-agent integrations, supply the key to `ssh` without writing it to disk in plaintext. Keys are never synced to cloud storage in the clear, and a separate key pair per machine keeps the blast radius of a compromised laptop local.

After this change, password-based login attempts are rejected before credentials are even evaluated, which eliminates brute-force and credential-stuffing as viable attacks against the machine. The exercise introduces `nano` as a terminal editor and `systemctl` as the interface to the `systemd` service manager, both of which return throughout the course.

Exercise: Provisioning Virtual Machines

INSTALL THE ORACLE VIRTUALBOX MANAGER for your host OS with default settings, and download two Ubuntu LTS ISOs: the Desktop edition for `dev` and the Server edition for `prod`. The asymmetry is deliberate, `dev` carries a graphical environment so you can run an editor and a browser locally, while `prod` stays headless because a production server has no business running a desktop. You can also decide to run dev on your machine directly, but this is not further supported in the excercises.

EXERCISES are for practice and reinforcing concepts. Work through them yourself first, break things, discuss, this is not a time trial. If your VM ends up in a state you cannot recover, restore the `B1-fresh` snapshot and start over. At the end of the session, take a snapshot named `B1-complete`, your safety net for the next block.

CREATE THE `dev` VM in the Oracle VirtualBox Manager. Click `New` and configure:

- Name `dev`, Type `Linux`, Version `Ubuntu (64-bit)`
- Memory at least 4096 MB, CPUs at least 2
- Hard disk at least 25 GB, VDI, dynamically allocated

WHERE TO GET THEM. VirtualBox: `https://www.oracle.com/virtualization/virtualbox/`. Ubuntu Desktop LTS ISO: `https://ubuntu.com/download/desktop`. Ubuntu Server LTS ISO: `https://ubuntu.com/download/server`. Pick the LTS release in both cases, not the interim one, so support extends across the semester.

CREATE THE `prod` VM with the same procedure but tighter resources, since it runs headless:

- Name `prod`, Type `Linux`, Version `Ubuntu (64-bit)`
- Memory at least 2048 MB, CPUs at least 2
- Hard disk at least 10 GB, VDI, dynamically allocated

INSTALLER CHOICES that matter: use distinct credentials per VM, `user/password` on `dev` and `produser/prodpassword` on `prod`. The asymmetry is deliberate, you should feel that logging into `prod` is a different act from working on `dev`.

ATTACH THE MATCHING ISO to each VM. VirtualBox offers an unattended installation checkbox during creation, don't select it, you want the interactive installer. Once a VM exists, select it in the manager and open `Settings`, `Storage`. Under `Storage Devices` you will see a controller, `IDE` or `SATA`, with an `Empty` optical drive beneath it. Click that `Empty` entry, then on the right under `Attributes` click the small disc icon next to `Optical Drive`, choose `Choose a disk file`, and pick the Desktop ISO for `dev` or the Server ISO for `prod`. Confirm with `OK`, start the VM, follow the installer, when in doubt go with the defaults and just forward-enter.

WHY THE SPREAD. Ubuntu Desktop with GNOME wants roughly 4 GB of RAM and 25 GB of disk to feel responsive. Ubuntu Server has no GUI and runs comfortably in a quarter of that. Sized this way the pair fits inside a 6 GB / 35 GB envelope on the host.

AFTER THE FIRST BOOT OF EACH VM (BOTH DEV AND PROD), INSTALL THE OPENSSH SERVER TO ENABLE REMOTE ACCESS:

```
sudo apt update
sudo apt install openssh-server
```

This step is required for both Desktop and Server editions. Reboot after installation.

HOST-ONLY NETWORK. In `File`, `Host Network Manager` (or `File`, `Tools`, `Network Manager` on version 7+), create a host-only network if none exists, typically 192.168.56.1/24. **Disable DHCP** for this network so you can assign static IPs manually.

Configure the network adapters for each VM: Select the VM in VirtualBox Manager and open `Settings`, then go to the `Network` section. For Adapter 1, check "Enable Network Adapter" and set it to `NAT` (for internet access). For Adapter 2, select the Adapter 2 tab, check "Enable Network Adapter," set it to `Host-only Adapter`, and choose the host-only network you created.

Missing Adapter 2? Power-off the VM, make sure you are in Expert-mode, go to `Settings`, `Network`, select Adapter 2, check "Enable Network Adapter," and set it to `Host-only Adapter`.

Assign static IPs inside each VM:

1. Boot the VM and log in.
2. Run `ip a` to list network interfaces. The host-only adapter is usually `enp0s8`, but confirm whether the name shows up in the output.
3. Edit the Netplan config with `sudo nano /etc/netplan/00-installer-config.yaml` Save and exit nano with (Ctrl+X, Y, Enter).
4. For `dev`, set the config as:

```
network:
  version: 2
  ethernets:
    enp0s8:
      addresses:
        - 192.168.56.10/24
```

5. Apply the config with `sudo netplan apply`
6. Check the new address with `ip a` again; you should see the static IP assigned to `enp0s8`.

Open a terminal (Desktop). On Ubuntu Desktop press `Ctrl+Alt+T`, or open `Activities` (top-left) and type "terminal"; press Enter. You can also right-click the desktop and select "Open Terminal" if the menu is present.

For `prod`, use `192.168.56.11/24` instead.

Netplan permission warning. Netplan will warn if a configuration file is writable by group/others or not owned by `root`. This is a security check to avoid loading untrusted configuration files; expected ownership is `root:root` and modes such as `644` for the YAML files.

Verify connectivity: On `dev`, run `ping 192.168.56.11` to test if `prod` responds. If you get no reply, double-check the Netplan configuration (correct interface and address), ensure Adapter 2 is set to Host-only in both VMs, and confirm that both VMs are running and attached to the same host-only network.

Troubleshooting. If the interface name is not `enp0s8`, use the correct one from `ip a`.

```
user@dev:~$ ping 192.168.56.11
PING 192.168.56.11 (192.168.56.11) 56(84) bytes of data.
64 bytes from 192.168.56.11: icmp_seq=1 ttl=64 time=1.63 ms
```

Figure 3: ping

Exercise: Navigating Linux

THIS EXERCISE turns the five commands and the permission model into something your hands remember. You walk the directories common to every Linux system, identify yourself to the machine, and read and change a file's mode string. By the end, the output of `ls -la` should read as a description you can speak aloud rather than a blob of symbols.

TEN-FINGER TYPING is part of the toolchain once the terminal is your main tool. A server has no menus, so your throughput is bounded by how quickly you can speak to the shell without looking at the keys; every hunt for a slash, tilde, or brace is attention spent on the keyboard instead of the problem. Ten minutes a day at `https://www.keybr.com/`, `https://typing.io/`, or `https://monkeytype.com/` is enough to remove that tax over a few weeks.

EXPLORE THE FILESYSTEM by visiting `/home`, `/etc`, and `/opt` in turn with `cd` and `ls -la`, and read two files with `cat`:

```
cat /etc/hostname
cat /etc/os-release
```

THE FILESYSTEM AS MAP. `/home` is where users live, `/etc` holds configuration, and `/opt` is where PixelWise will be installed in a later block. These locations reflect decades of convention you will meet on every Linux machine; knowing them lets you locate things by instinct.

CREATE A WORKSPACE and check that you are where you think you are:

```
mkdir ~/workshop && cd ~/workshop
pwd
touch notes.txt
rm notes.txt
```

`pwd` is a reassurance command. When something does not behave as you expect, asking the shell where it thinks it is usually explains the problem faster than re-reading the error.

UNDERSTAND WHO YOU ARE and look at the permissions on your new file:

```
whoami
id
ls -la notes.txt
```

Decode the first column of `ls -la` against the margin note on mode strings from Block 2. Name aloud what each of the ten characters means; if you can do that without looking, permissions have stopped being mysterious.

CHANGE THE MODE AND WATCH IT:

```
chmod 600 notes.txt
ls -la notes.txt
```

The next exercise will require `600` on your SSH private key, or OpenSSH refuses to use it; you have now seen what `600` looks like and why ownership matters.

A THOUGHT EXPERIMENT ON LEAST PRIVILEGE. Your `user` account can read `/etc/os-release` but cannot edit `/etc/ssh/sshd_config` without `sudo`. Why does that separation exist, and what would change if the web service we deploy in a later block ran as `root`? Name one file an attacker who compromised such a service could reach that a non-root service could not; the mental habit matters more than the exact file.

Exercise: SSH and Key-Based Access

This exercise walks through three states of remote access in order: password login, key-based login, and password login disabled. The point is not the commands but to recognise which state you are in at any moment and why each is stronger than the last.

First, log in with a password. From `dev`, open a session to `prod`:

```
ssh produser@192.168.56.11
```

Type your password, look around briefly, and log out with `exit`. This is the baseline. Every later step is interpretable as an upgrade over this state.

Generate a key pair on `dev` and install it on `prod`:

```
ssh-keygen -t ed25519 -C "you@example.com"
ssh-copy-id produser@192.168.56.11
```

Reconnect with `ssh produser@192.168.56.11` and notice the password prompt is gone. Log in and inspect what `ssh-copy-id` wrote for you:

```
cat ~/.ssh/authorized_keys
```

What just happened. `ssh-copy-id` appended the contents of `id_ed25519.pub` on `dev` to `~/.ssh/authorized_keys` on `prod`. At the next connection the daemon issued a challenge that only the private key on `dev` could answer; no secret material ever left the client. Seeing your public key in a readable file anchors the whole exchange to something concrete.

Disable password authentication. Edit `/etc/ssh/sshd_config` with `sudo nano`, set `PasswordAuthentication no`, and restart the daemon:

```
sudo systemctl restart ssh
sudo systemctl status ssh
```

The `status` output should show `active (running)`. If it does not, keep your current session open and fix the configuration before logging out; a locked-out VM is rescued by restoring the snapshot, not by wishful thinking. From a machine without the key, the daemon now rejects the attempt without even asking for a password.

Take a snapshot of both VMs once everything works, naming them `B1`; this is your known good state for the rest of the course. Then deliberately break something inside a VM, watch the fallout, and restore the snapshot. You now have physical evidence that the safety net holds.

Nuke the machine. From `prod`, run `sudo rm -rf /` to delete all files on the machine. Restore from the snapshot.

Run prod without GUI. In future run prod headless without GUI from the VirtualBox Manager. You can still log in with SSH and run commands, but there is no desktop to interact with. This is how a real production server works.

Self-Reflection and Recap

SELF-REFLECTION Questions which can guide your thoughts during and after the exercises:

- Why do we use two separate machines instead of running everything on one?
- Why is disabling password authentication a stronger protection than picking a longer password?
- What does the principle of least privilege mean in the context of file ownership, and where did it show up in today's workshop?
- If your laptop died tomorrow, how much of today's setup could you reproduce, and what would you lose?
- Which of the five Linux commands do you still have to look up, and which one surprised you?
- How does the two-machine model map to a real cloud deployment with staging and production? How is our development machine different from staging?

RECAP of Key Concepts:

- Virtual machines isolate experiments from the host and make snapshots a first-class safety net.
- The two-machine architecture separates development from production at the hardware level.
- Files on Linux carry owners, groups, and three sets of permissions, and `chmod` and `chown` manage both.
- SSH is the only door into the server, and key-based authentication replaces passwords with cryptography that cannot be guessed.

MILESTONE. So far, we own the machine. The next step is to own the way we build software on it. In the next chapter we introduce Git and GitHub: not just a version history, but the collaboration layer that lets a team branch, review, and merge work in parallel. It is the tool that turns two developers editing the same file from a disaster into a workflow.

A NOTE ON WHAT COMES NEXT. The commands you ran in this session are reproducible but not yet reproduced. You ran them in a terminal, they worked, and they are gone. In the next block we place them under version control and turn the sequence into a script that any future VM can run end to end, without you remembering anything.

WITHOUT SHIPPING ANYTHING YET, we already have two machines that talk to each other, a secure door, and a known good state to fall back on.

TEASER. You and a classmate both want to improve PixelWise at the same time. How do you work on the same codebase without overwriting each other's changes, review what the other person did, and merge the results safely?

Version Everything: Git & GitHub

Git a Life.

The Why

Block 1 gave us two machines and a way to reach one from the other. We can write code on `dev` and, eventually, deploy it to `prod`. But nothing yet records what changed between deployments, who changed it, or how to undo a mistake. The moment two people touch the same file, the same function, or one experiment needs to coexist with another, the workflow breaks down entirely.

PixelWise needs to live on two machines. Right now there is no reliable way to move code from `dev` to `prod` and keep both in sync. Git gives us exactly that: a shared history that either machine can pull from, so deployment becomes a deliberate, testable, repeatable step instead of manual file copying.

Version control solves an entire family of problems at once. It records every change with a message, a timestamp, and an author, so the full history of a project is always available. It lets you return to any previous state in seconds, turning risky experiments into cheap, reversible ones.

Parallel development becomes natural. Multiple people work on the same codebase without overwriting each other's changes. A new feature, a bug fix, and a refactor can all proceed simultaneously on isolated branches and be integrated later with explicit, reviewable merge steps.

Downtime shrinks as a direct consequence. Developers merge through a guided review step that catches conflicts and errors before they reach `main`, deploying only code that has already been integrated and verified. When something still slips through, you can roll back to the last known-good state in a single command instead of debugging under pressure while users wait.

Combined with a hosting platform like GitHub, version control becomes the backbone of code review, continuous integration,

GitHub: https://github.com

issue tracking, and project management, built around the software development process.

Hands On Experience

CONSIDER THIS SCENARIO: you and a classmate are both improving the PixelWise classifier. You rewrite the data loader; your classmate changes the model architecture. You each work on your own laptop for a few hours, then try to combine the results. Without a system for tracking changes, combining means copying files back and forth, comparing line by line, and hoping nothing was lost. Worse still, if both of you would have worked on the same machine, overwriting each others files as you go.

GIT IS THAT SYSTEM. It tracks exactly what changed in every file, when, and by whom. It lets you branch off to try an idea, or build a feature without disturbing stable code, merge branches back together with full visibility into conflicts, and review each other's contributions before they reach the shared codebase. The design principle is simple: never lose work, and never wonder what happened.

EVERY TEAM that ships software uses version control. It is not an optional workflow preference; it is the infrastructure that makes collaborative, iterative development possible at any scale. As projects grows, interface contracts, clear agreements on inputs, outputs, and responsibilities between components, will become increasingly important. Version control lays the foundation by making every change visible and reviewable, so contracts can be enforced rather than assumed.

THIS SECOND BLOCK introduces Git and GitHub as the version control layer for PixelWise. By the end you will have

- understood the Git data model and why it makes history tamper-proof,
- practised the staging workflow of editing, staging, and committing changes,
- created branches, merged them, and resolved conflicts,
- pushed code to a remote repository on GitHub and collaborated through pull requests,
- and configured SSH-based authentication so Git operations are seamless from both VMs.

The Git Data Model

GIT IS NOT A BACKUP TOOL. It is a content-addressable filesystem with a version history layered on top. Understanding the data model makes every command intuitive.

Git was created by Linus Torvalds in 2005 to manage the Linux kernel source code. It is now the de facto standard for version control in software development.
Git: `https://git-scm.com/`
Pro Git book (free): `https://git-scm.com/book/en/v2`

Blob A file's contents, stored by its SHA-1 hash. The name is irrelevant; only the content matters.

Tree A directory listing: maps filenames to blobs (files) or other trees (subdirectories).

Commit A snapshot of the entire project at a point in time. Contains: a pointer to the root tree, the author, a timestamp, a message, and a pointer to the parent commit(s).

Branch A lightweight, movable pointer to a commit. `main` is a branch. Creating a branch is instantaneous; it just creates a new pointer.

HEAD A pointer to the branch you are currently on. When you commit, `HEAD` moves forward.

EVERY COMMIT is identified by a SHA-1 hash, a 40-character hexadecimal string like `a3f2b7c...`. This hash is computed from the commit's contents. If anything changes, even a single character in a single file, the hash changes. This makes Git history tamper-proof.

DATA HYGIENE follows directly from the data model. Because Git stores every version of every blob permanently, committing a large dataset means carrying it in the repository history forever, even if you delete the file later. A 500 MB training set committed once, modified twice, and then removed still occupies 1.5 GB of history. Repositories should contain code, configuration, and small metadata files. Large or frequently changing binary assets, datasets, and model weights do not belong in Git.

The `.gitignore` file, covered in its own subsection below, is the practical mechanism for keeping these artefacts out of the repository.

DEDICATED TOOLS fill the gap. Git LFS replaces large files with lightweight pointers. Hugging Face Hub hosts datasets and model weights with built-in Git LFS. DVC tracks data pipelines and links datasets to experiments. Weights & Biases versions datasets alongside experiment logs.
Git LFS: `https://git-lfs.com`
Hugging Face Hub: `https://huggingface.co`
DVC: `https://dvc.org`
W&B: `https://wandb.ai`

The Staging Workflow

GIT HAS THREE AREAS:

- *Working directory* The files on disk. What you edit.
- *Staging area* What you have marked for the next commit. A deliberate selection.
- *Repository* The committed history. Permanent, immutable snapshots.

THE NAPKIN CYCLE as easy as it gets:

```
git pull
git add <files>
git commit -m "..."
git push
```

Pull the latest changes, stage your work, commit it, push it. Every other command is a variation on this loop.

THE CORE COMMANDS:

```
git status              # what changed? what is staged?
git add file.py         # stage a file for the next commit
git add .               # stage everything (use with care)
git commit -m "message" # commit the staged changes
git diff                # unstaged changes vs last commit
git diff --staged       # staged changes vs last commit
git log                 # show commit history
git log --oneline       # compact view: one line per commit
```

Write commit messages in the imperative mood: "Add classifier module" not "Added classifier module." The message completes the sentence "This commit will . . . ". Keep the first line under 72 characters. For a gallery of how not to do it, visit https://whatthecommit.com.

WHEN TO COMMIT? Commit when you have completed one logical change: a bug fix, a new function, a config update. If you struggle to write a short commit message, you probably changed too many things at once. If you go hours without committing, you are accumulating risk. Small, frequent commits make history useful; large, rare commits make it a wall of text.

```
(\  /)
 (O.o)
 (>  <)
```

Bunny approves these changes.

Branching and Merging

BRANCHES LET YOU WORK ON IDEAS IN ISOLATION. The `main` branch is the stable baseline. Feature branches diverge, evolve, and merge back when ready.

```
git branch                  # list branches
git branch feature-x        # create a new branch
git checkout feature-x      # switch to it
git checkout -b feature-y   # create and switch in one step
git branch -d feature-x     # delete after merge
```

Delete branches after merging. Stale branches accumulate fast and make `git branch` unreadable. If the branch is merged, it is safe to delete; the history lives on in `main`.

WHEN THE FEATURE IS READY, merge it back and clean up:

```
git checkout main
git merge feature-x
git branch -d feature-x
```

Merge Conflicts

A MERGE CONFLICT occurs when two branches changed the same line in the same file. Git cannot decide which version to keep, so you must resolve it manually.

GIT MARKS THE CONFLICT in the file. The block between `«««< HEAD` and `=======` is what the current branch has; `HEAD` always means "the commit you are on right now." The block between `=======` and `»»»> feature-x` is what the incoming branch has.

```
<<<<<<< HEAD
result = model.predict(x)
=======
result = classifier.classify(x)
>>>>>>> feature-x
```

Edit the file to keep the correct version and remove the markers. Then stage the resolved file with `git add file.py` and commit with `git commit -m "Resolve merge conflict in file.py"`.

Tags

A TAG MARKS A POINT IN HISTORY THAT MATTERS: a release, a milestone, a known-good state.

```
git tag v0.1                          # lightweight tag
git tag -a v0.1 -m "initial setup"    # annotated tag (preferred)
git push --tags                       # push tags to remote
```

Lightweight tags are just a name pointing to a commit. Annotated tags store the tagger, date, and message; use them for releases. We tag `v0.1` today; the tag becomes meaningful when you look back at it in Block 8.

`tig` is a terminal-based Git browser that makes it easy to navigate commits, branches, and tags on a headless Ubuntu machine. Install with `sudo apt install tig`.
https://jonas.github.io/tig/

Semantic Versioning: https://semver.org

SEMANTIC VERSIONING gives tag names a shared meaning. A version number follows the pattern `MAJOR.MINOR.PATCH`. Increment `PATCH` for bug fixes that do not change behaviour, `MINOR` for new features that remain backwards-compatible, and `MAJOR` when you introduce breaking changes. A jump from `v1.2.3` to `v1.3.0` tells users that something was added but nothing they depend on was removed. A jump to `v2.0.0` warns them to check for incompatibilities.

Remotes and GitHub

```
git clone <url>              # copy a remote repo locally
# fork: copy repo to your GitHub account (GitHub UI, not a git command)
git remote add origin git@github.com:user/pixelwise.git
```

A REMOTE is a copy of your repository on another machine, usually GitHub.

CLONE AND FORK are two ways to start from an existing repository.

A `fork` is a GitHub concept, not a Git command. It creates a full copy of someone else's repository under your own GitHub account. You clone your fork locally, work on it freely, and propose changes back to the original via a pull request. Forking is the standard workflow for contributing to open-source projects where you do not have write access to the original repository.

`git clone` copies a remote repository to your local machine, keeping the original as the `origin` remote. You can pull updates and, if you have permission, push changes back.

GitHub is not Git. Git is the version control system; GitHub is a hosting platform that adds pull requests, issues, CI/CD, and collaboration features on top of Git. Alternatives include GitLab and Bitbucket.
GitHub: https://github.com/
GitHub Docs: https://docs.github.com/

Pull Requests

PUSH sends your local commits to the remote. Until you push, your work exists only on your machine.

FETCH does the opposite: it downloads new commits from the remote but leaves your working directory untouched, so you can inspect what changed before merging.

PULL combines fetch and merge in one step, updating your local branch to include the remote's latest commits.

```
git push -u origin main     # push and set upstream
git pull                     # fetch + merge from remote
git fetch                    # fetch without merging
```

THE `-u` FLAG links your local branch to the remote branch. After running `git push -u origin main` once, Git remembers the connection and future `git push` and `git pull` work without specifying the remote or branch.

A PULL REQUEST is a proposal to merge a branch into another branch, typically a feature branch into `main`. It is the standard mechanism for code review: your changes are visible, reviewable, and discussable before they become part of the main codebase.

PULL REQUESTS AND CI. Because a PR represents a well-defined set of changes that has not yet reached `main`, it is the ideal moment to run a test suite, a linter, or any other check automatically. If the tests fail, the merge is blocked and `main` stays healthy. We will wire up this automation in a later block; for now, recognise that the PR is where human review and machine verification meet.

The `.gitignore` File

NOT EVERYTHING BELONGS IN VERSION CONTROL. The `.gitignore` file tells Git which files and directories to exclude from tracking:

```
# Python
__pycache__/
*.pyc
.venv/

# Secrets
.env

# Data and models (too large, reproducible)
data/
models/*.pkl

# Editor files
.vscode/
*.swp
```

THIS INCLUDES ALL CREDENTIALS: API keys, database passwords, webhook secrets, tokens for AI services. These live on the server in environment variables or protected configuration files, never in the repository. A leaked production database password or API key can be exploited within minutes by automated scanners.

Git history is permanent. A secret committed once is leaked forever, even if you delete the file in a later commit. The old commit still contains it. `.gitignore` is your first line of defence.

Exercise: Setting Up Git and the Repository

CREATE A GITHUB ACCOUNT if you do not have one. Then open a terminal on the `dev` VM.

INSTALL GIT. Ubuntu does not ship with Git by default, so install it through the package manager. It does not matter which directory you run this in, `apt` installs system-wide:

```
sudo apt update
sudo apt install -y git
git --version              # confirm the install succeeded
```

CONFIGURE YOUR GIT IDENTITY:

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

GENERATE AN SSH KEY. SSH keys authenticate you to GitHub from the command line, replacing the password prompt on every push and pull. Before generating, check whether a key already exists so you do not overwrite one you rely on for another server:

```
ls ~/.ssh/
```

If you see a file called `id_ed25519`, that path is already in use. Either reuse it for GitHub as well, or generate a new key under a distinct filename so both keep working. Otherwise, generate the default key pair on the `dev` VM:

```
ssh-keygen -t ed25519 -C "your@email.com"
```

The tool prompts for a file location. If `~/.ssh/id_ed25519` does not yet exist, press Enter to accept the default; if it does, type a distinct path such as `~/.ssh/id_ed25519_github` so the existing key is preserved. It then prompts for a passphrase, optional but recommended, it encrypts the private key on disk so a stolen key file is not immediately usable.

ADD THE PUBLIC KEY TO GITHUB. Print it to the terminal and select the entire output, beginning with `ssh-ed25519` and ending with the comment:

```
cat ~/.ssh/id_ed25519.pub
```

AT THE END OF THE SESSIONS, take a snapshot named `B2-complete`, your safety net for the next block.

OPENING A TERMINAL ON UBUNTU. The fastest way is the keyboard shortcut `Ctrl+Alt+T`, which launches the default terminal emulator. Alternatives: press the `Super` key (the Windows key) and type "terminal", or right-click on the desktop and choose "Open Terminal". Inside a terminal you can open another tab with `Ctrl+Shift+T` and a fresh window with `Ctrl+Alt+T` again.

`sudo` runs the command with administrator privileges, which `apt` needs to write to system directories. You will be asked for your password the first time. `apt update` refreshes the package index so you get the current version; `apt install -y` installs without prompting for confirmation.

RINGS A BELL? You generated an `id_ed25519` key in Block 1 to log in to the `prod` VM without a password. That same key is sitting in `~/.ssh/` now, which is exactly why `ls` matters before the next step: accepting the default path would overwrite the production key and lock you out of the server.

MULTIPLE SSH KEYS? If you keep one key for production servers and another for GitHub, tell SSH which to use for each host with an `~/.ssh/config` entry:

```
Host github.com
  HostName github.com
  User git
  IdentityFile
~/.ssh/id_ed25519_github
  IdentitiesOnly yes
```

Then `chmod 600 ~/.ssh/config`. Without `IdentitiesOnly`, SSH may offer the wrong key first and GitHub will reject the connection after a handful of auth failures.

WHAT JUST HAPPENED? `ssh-keygen` created two files in `~/.ssh/`. `id_ed25519` is the private key: never share it, never commit it, never copy it to another machine without good reason. `id_ed25519.pub` is the public key, safe to share, this is the file you paste into GitHub. The `-t ed25519` flag picks a modern signature algorithm;

In GitHub, navigate to Settings, then SSH and GPG keys, then New SSH key. Give it a recognisable title like `pixelwise-dev-vm` so you know which machine it belongs to, paste the public key into the Key field, and save.

TEST THE CONNECTION:

```
ssh -T git@github.com
```

The first time, SSH asks whether to trust GitHub's host fingerprint. Type `yes`. If the key is set up correctly, GitHub responds with `Hi <username>! You've successfully authenticated, but GitHub does not provide shell access.` The message about shell access sounds like an error but is the intended response: GitHub only allows Git operations over SSH, not interactive shells.

INITIALISE THE PIXELWISE REPO on the `dev` VM. Pick a stable working directory, your home folder is the natural place, so the project does not get lost in `/tmp` or buried under `Downloads`:

```
cd ~                          # start from your home directory
mkdir pixelwise && cd pixelwise
pwd                           # confirm: /home/<user>/pixelwise
git init
```

`cd ~` jumps to your home directory from anywhere. `pwd` prints the current path so you can confirm where you are before running `git init`. Avoid initialising a repo inside `/tmp`, on a mounted USB drive, or inside another existing Git repository.

Write a `README.md` with a one-line project description.

Exercise: Staging, Committing, and .gitignore

WRITE A `.gitignore`. Create a `.gitignore` file that excludes Python caches, virtual environments, secret files, dataset directories, model checkpoints, and editor files. Verify it works: create a file called `scratch.pyc`, run `git status`, and confirm Git does not list it. Then remove the test file.

MAKE YOUR FIRST COMMIT. Stage and commit `README.md` and `.gitignore`:

```
git add README.md .gitignore
git commit -m "Add project README and .gitignore"
```

Run `git log` to see your first commit. Run `git status` to confirm the working directory is clean.

THE HABIT TO BUILD: after every commit, run `git status` and `git log --oneline` to confirm what just happened. It takes two seconds and prevents surprises.

Exercise: Branching, Merging, and Conflicts

CREATE A FEATURE BRANCH and make a change to `README.md`:

```
git checkout -b feature-readme
nano README.md          # add a project description line
git add README.md
git commit -m "Add project description to README"
```

EDITING FILES IN THE TERMINAL. `nano README.md` opens the file in a simple terminal editor. The bar at the bottom of the screen lists the shortcuts, where ˆ stands for the `Ctrl` key: `Ctrl+O` writes the file to disk, `Ctrl+X` exits, and `Ctrl+G` shows full help. Vim and VS Code over Remote SSH work just as well if you prefer them.

SWITCH BACK TO `main` and edit the same line differently:

```
git checkout main
nano README.md          # edit the same line, but differently
git add README.md
git commit -m "Add alternative description to README"
```

`main` OR `master`? Older Git versions called the default branch `master`. Since 2020, the convention has shifted to `main`, and GitHub creates new repositories with `main` by default. If `git checkout main` reports that the branch does not exist, run `git branch` to see what your repo actually calls it. To bring an existing repository in line with the new convention, rename in place with `git branch -m master main`, then set the default for future `git init` runs with `git config --global init.defaultBranch main`.

MERGE AND RESOLVE THE CONFLICT. Now merge the feature branch and observe the conflict:

```
git merge feature-readme
```

Git reports a conflict and pauses the merge. Open `README.md` with `nano README.md`. You will see a block that looks like this:

```
<<<<<<< HEAD
... your text on main
=======
... text from feature-readme
>>>>>>> feature-readme
```

`HEAD` marks the version sitting on the branch you are currently on, in this case `main`. The lines after `=======` are what `feature-readme` is bringing in. Choosing the correct version means deciding what the file should look like once both changes are reconciled. You have three options: keep only the `HEAD` side, keep only the incoming side, or write a merged version that combines parts of both. Then delete all three marker lines, `«««< HEAD`, `=======`, and `»»»> feature-readme`, so the file contains only the resolved content. Save with `Ctrl+O` and exit with `Ctrl+X`, then complete the merge:

`git status` DURING A MERGE. While the merge is paused, `git status` reports "You have unmerged paths" and lists every file with conflict markers still in it. Run it any time to remind yourself which files still need to be resolved before you can finish the merge.

```
git add README.md
git commit -m "Resolve merge conflict in README"
```

Verify with `git log --oneline --graph` that both branches are now integrated.

Exercise: Tags and History Browsing

TAG THE RELEASE. Mark this point in history:

```
git tag -a v0.1 -m "initial server setup"
```

Verify the tag exists with `git tag -l` and inspect it with `git show v0.1`.

INSTALL `tig`, a terminal-based Git browser:

```
sudo apt install -y tig
tig
```

Navigate the commit history and find your tag. Press `q` to quit.

WHY NO `git push` YET? Tags live locally until you push them. We have not configured a remote yet, that happens in the next exercise. Once the remote exists, `git push --tags` ships every local tag to GitHub, where it appears under the repository's "Releases" tab.

VS CODE can also browse Git history visually. Open the PixelWise folder via `code .` or Remote SSH, then use the Source Control panel and the GitLens extension to explore commits, branches, and tags with a graphical interface.

Exercise: Remotes, Push, and Clone

CONNECT TO GITHUB. Create a repository on GitHub, add it as a remote, push the commits, then push the tag you created in the previous exercise:

```
git remote add origin git@github.com:user/pixelwise.git
git push -u origin main
git push --tags
```

Verify on GitHub that your commits, `.gitignore`, and the `v0.1` tag under the "Releases" tab are all visible.

REPLACE `user` WITH YOUR GITHUB USERNAME. The `user` placeholder in the remote URL is just a stand-in. Substitute your own GitHub handle, the one shown in your profile URL at `github.com/<your-handle>`, so the URL points at the repository under your account. The same goes for the `git clone` URL on the server below. Pushing to a `user/pixelwise` that you do not own will fail.

CLONE TO THE SERVER. SSH into the server and clone PixelWise to its permanent home:

```
sudo mkdir -p /opt/pixelwise
sudo chown produser:produser /opt/pixelwise
git clone https://github.com/user/pixelwise.git /opt/pixelwise
```

PixelWise now lives on both machines. Run `git log --oneline` on the server to confirm the full history arrived.

SSH INTO `prod` AGAIN. From the `dev` VM, open a session to `prod` the same way you did in Block 1:

```
ssh produser@192.168.56.11
```

The key-based login from Block 1 is still in place, no password should be required. Once you are on `prod`, the prompt changes to `produser@prod`, and the commands below run on the server. Type `exit` when you are done to return to `dev`.

WHY HTTPS ON `prod`, SSH ON `dev`? `dev` pushes commits back to GitHub, so it needs the SSH key you registered earlier. `prod` only ever pulls, it is a deployment target, never an authoring machine. HTTPS clone of a public repository requires no key, no token, no extra setup, which matches how real deployment servers usually have read-only access to the source repository. If your repository is private or you later need to push from `prod`, generate a second SSH key on `prod` and register it on GitHub the same way you did on `dev`.

Self-Reflection and Recap

Reflection questions:

- Why does changing a single character in a file produce a completely different commit hash?
- What is the difference between `git add` and `git commit`, and why does Git separate the two steps?
- When would you use a branch instead of committing directly to `main`, and what happens when a merge produces a conflict?
- Why should you never commit `.env` files or API keys, and what is the role of pull requests in catching mistakes before they reach `main`?
- Why does PixelWise use SSH keys rather than passwords for Git operations across both VMs?

Milestone: Your project lives on GitHub with a README, a `.gitignore`, and a tagged release. The server has the repo cloned at `/opt/pixelwise`. Both machines share the same history. In the next block we turn the manual package installs into a reproducible setup script and tackle dependency management.

Dependencies & Structure

Works on my machine.

The Why

Block 2 left us with a PixelWise repository on GitHub and a server that has Git and Python installed. But those packages were installed by hand, one `sudo apt install` at a time. If the VM is rebuilt, or a new team member joins, nobody remembers what to install or in which order.

PixelWise now lives on GitHub and on both machines. But the repository contains only a README and a `.gitignore`. Before we can write application code, we need a reproducible way to install everything the project depends on, from system packages to Python libraries, so that `dev` and `prod` stay in sync without human memory.

"Works on my machine" is the most dangerous sentence in software engineering. Your colleague clones the repo, runs the code, and gets an `ImportError`. You installed `scikit-learn` months ago and forgot. They did not. The code is correct; the environment is wrong.

Dependency management solves this at two levels: system packages and Python packages. For system packages, we capture every `apt install` in a shell script called `setup-server.sh` so any fresh VM can be provisioned identically. For Python packages, we declare what the project needs in a `requirements.txt`, pin the exact versions, and anyone can reconstruct the identical environment in two commands.

Hands On Experience

Consider this scenario: you hand your PixelWise repo to a classmate. They clone it, type `python3 train.py`, and immediately hit `ModuleNotFoundError: No module named 'sklearn'`. You tell them to install scikit-learn. They do, but get version 2.0 while you wrote the code against 1.4. The training script crashes with a different error. Now you are debugging version mismatches instead of building features.

The two-command promise. A well-managed project lets any new developer go from clone to running code with exactly two commands: create the virtual environment and install from a pinned requirements file. No guessing, no Slack messages asking "which version of numpy do I need?"

This third block introduces dependency management and project structure for PixelWise. By the end you will have

- captured system-level setup in a reproducible shell script,
- created and activated a Python virtual environment,
- installed packages with pip and pinned them in `requirements.txt`,
- separated secrets from code using `.env` and `.env.example`,
- scaffolded the PixelWise project directory,
- and audited your dependencies for known security vulnerabilities.

The Setup Script

On the server VM, you installed packages manually in Block 1 and Block 2. Now capture that work in a script so any fresh VM can be provisioned identically:

```
#!/bin/bash
# setup-server.sh: provision a fresh server VM
set -euo pipefail

sudo apt update
sudo apt install -y git python3 python3-pip \
    python3-venv curl
```

The `set -euo pipefail` line makes the script fail immediately on any error (`-e`), on undefined variables (`-u`), and on pipe failures (`-o pipefail`). This prevents silent failures that leave the server in a half-configured state.

Mark it executable and run it:

```
chmod +x setup-server.sh
./setup-server.sh
```

This is version control paying for itself. The script is committed to Git, so when a new team member joins or a VM is rebuilt, `bash setup-server.sh` reproduces the exact same environment. No Slack messages, no guessing.

As the project grows, this script may split into multiple files or graduate to a configuration management tool like Ansible. The principle stays the same: every setup step lives in version control.
Ansible: https://docs.ansible.com/
GNU Make: https://www.gnu.org/software/make/

System packages vs. Python packages. The setup script handles system-level dependencies: Git, Python itself, system libraries that compiled Python packages need. Python-level dependencies live in `requirements.txt` and are managed by pip inside a virtual environment. The two layers are complementary: the setup script prepares the ground, the virtual environment grows the project.

Virtual Environments

A virtual environment is an isolated Python installation. Each project gets its own set of packages, independent of every other project and the system Python.

```
python3 -m venv .venv
source .venv/bin/activate
```

The `.venv/` directory contains a full Python installation: interpreter, `pip`, and all installed packages. It is machine-specific and large, often 50 to 200 MB. It must be in `.gitignore`: never commit it. Anyone can recreate it from `requirements.txt`.

While the environment is active, `python` and `pip` point to the local copies. Install a package and it stays inside `.venv/`, with no system-wide pollution. Deactivate with `deactivate`.

Verify the switch:

```
which python
# /home/user/pixelwise/.venv/bin/python

pip list
# Package    Version
# ---------- -------
# pip        24.0
# setuptools 69.5.1
```

A fresh environment starts empty. Every package you install from this point is tracked and intentional.

The data science ecosystem often uses `conda` instead of `venv`. Conda manages both Python packages and system-level libraries like CUDA or MKL, which makes it popular for GPU workflows. For web projects, `venv` is the lighter and more standard choice.
Conda: https://docs.conda.io/

The rule is simple: every project gets its own `.venv`. Project A needs `numpy==1.24`; Project B needs `numpy==2.0`. Without isolation, one of them breaks. With virtual environments, they coexist peacefully on the same machine.

Managing Packages with pip

pip is the package installer for Python. The core cycle:

```
pip install scikit-learn joblib python-dotenv
pip freeze > requirements.txt
pip install -r requirements.txt
```

- `pip install` downloads and installs packages from PyPI (the Python Package Index): https://pypi.org/.
- `pip freeze` lists every installed package with its exact version.
- `pip install -r` installs everything listed in a requirements file, the reproducibility step.

Anatomy of `requirements.txt`

One package per line, pinned to an exact version:

```
scikit-learn==1.4.0
joblib==1.3.2
python-dotenv==1.0.1
```

Version specifiers: == pins exactly, >= sets a floor, ~= allows patch updates (~=1.4.0 means >=1.4.0, <1.5.0). For maximum reproducibility, pin with ==.

Semantic versioning: https://semver.org

Configuration: `.env` and `python-dotenv`

SEPARATING CONFIGURATION FROM CODE is a foundational principle. The 12-factor app methodology states: store config in the environment, never in the source code.

The Twelve-Factor App: https://12factor.net/

python-dotenv: https://github.com/theskumar/python-dotenv

A `.env` FILE holds key-value pairs that your application reads at startup:

```
SECRET_API_KEY=my-actual-secret-key-here
DEBUG=true
```

In Python, `python-dotenv` loads these into environment variables:

```
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("SECRET_API_KEY")
debug = os.getenv("DEBUG", "false").lower() == "true"
```

The `.env.example` Pattern

THE `.env` FILE CONTAINS REAL SECRETS, so it must never be committed. The `.env.example` file is the contract: it shows which variables exist, what they are for, and provides safe placeholder values:

```
# API key callers must send in the X-API-Key header
# (a secret, never a real value here)
SECRET_API_KEY=replace-me

# Debug mode: when true, FastAPI shows /docs and full
# error tracebacks. Never true in production.
DEBUG=true
```

`.env.example` is committed to Git and public. `.env` is in `.gitignore` and stays local. The contrast is immediate: one is the template, the other is the secret. This pattern grows block by block, every new service adds its variables to `.env.example`.

STUDENTS CREATE THEIR ACTUAL `.env` by copying `.env.example` and replacing the placeholders with real values.

SECURITY AUDITING. Your dependencies have their own dependencies, and any of them can harbor known vulnerabilities. `pip-audit` checks every installed package against the CVE database: `pip install pip-audit && pip-audit`. Run it after every `pip freeze`. In Block 8 we automate this with a cron job so vulnerabilities surface without human memory.

`pip-audit`: https://github.com/pypa/pip-audit

Dependabot: https://docs.github.com/en/code-security/dependabot

The Container Alternative

Docker and Docker Compose solve the "reproducible environment" problem at the OS level rather than the Python level. This course teaches the virtualenv path to keep the OS layer visible and learnable. Docker is the natural next step after this course.
Docker: https://www.docker.com/
Docker Docs: https://docs.docker.com/get-started/

VIRTUAL ENVIRONMENTS ISOLATE PYTHON PACKAGES. Docker containers isolate the entire operating system: libraries, system packages, Python version, everything. Where `requirements.txt` says "install these Python packages," a `Dockerfile` says "start from this Linux image, install these system packages, then install these Python packages."

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "main.py"]
```

The result is a self-contained unit that runs identically on any machine with Docker installed. We do not use Docker in this course, but you should know it exists and why teams adopt it.

Optional: When `requirements.txt` Starts to Hurt

Tools like `uv` and `poetry` use this model and are fast becoming the standard. Once the `requirements.txt` workflow is second nature, this is the upgrade.
https://packaging.python.org/en/latest/

`pip freeze > requirements.txt` captures everything: your direct dependencies and all their transitive dependencies, flattened into one list. Six months later you want to upgrade one package: which of those 47 lines do you actually own? `pyproject.toml` solves this by separating *what you depend on*, your direct deps loosely pinned, from *what gets installed*, a generated lock file with the full resolved graph. Upgrades become surgical: bump one line, regenerate the lock, done.

```
[project]
name = "pixelwise"
version = "0.1.0"
dependencies = [
    "scikit-learn>=1.4",
    "joblib>=1.3",
    "python-dotenv>=1.0",
]
```

COMPARE THIS TO `requirements.txt`: the project file lists only what you chose to install, with flexible version bounds. The tool resolves the full dependency graph and writes a lock file that pins

every transitive dependency. You commit both files; the lock file is your reproducibility guarantee.

Exercise: The Setup Script

Exercises are for practice and reinforcing concepts. Work through them yourself first, break things, discuss, this is not a time trial. If your VM ends up in a state you cannot recover, restore your own `B2-complete` VM snapshot or follow the tag catch-up below. At the end of the session, take a snapshot named `B3-complete`, your safety net for the next block.

Catching up via the reference tag. If you joined late, missed the previous block, or your VM is in a state you cannot recover, you do not need to redo every prior exercise. The course maintains a public reference repository at `https://github.com/schutera/pixelwise` with a tag at the end of every block. Restore the distributed `B1-complete` VM image, complete the Block 2 Git identity and SSH key setup so the VM has your credentials, then clone the reference repository at the `v0.1` tag, which marks the end of Block 2:

```
cd ~
git clone --branch v0.1 https://github.com/schutera/pixelwise.git
cd pixelwise
git switch -c main
git remote set-url origin git@github.com:<you>/pixelwise.git
```

You are now at the state where Block 2 ended, ready to start Block 3.

Why a tag, not a VM snapshot? The reference repository carries a tag at the end of every block: `v0.1` for Block 2, `v0.2` for Block 3, and so on, all visible at `https://github.com/schutera/pixelwise/tags`. Each tag points at a specific commit, so checking out a tag rewinds the working tree to the exact state the block ended on. The tag carries no credentials, only the project history. You then repoint `origin` at your own GitHub repository so future pushes go to your account, not the reference one. From this block onwards, `setup-server.sh` and `requirements.txt` reproduce the system and Python environments on top, so a fresh VM goes from empty to running by combining the `B1-complete` image, the reference tag, and the scripts.

Write `setup-server.sh`. On the `dev` VM, inside the PixelWise repo, create a shell script that captures the system packages you installed manually in earlier blocks. Add `set -euo pipefail` at the top so the script fails loudly on any error.

```
#!/bin/bash
set -euo pipefail

sudo apt update
sudo apt install -y git python3 python3-pip \
    python3-venv curl
```

Commit it. This is the first file that makes provisioning reproducible:

```
git add setup-server.sh
git commit -m "Add server provisioning script"
git push
```

Exercise: Virtual Environments and pip

CREATE AND ACTIVATE A VIRTUAL ENVIRONMENT on the `dev` VM:

```
cd ~/pixelwise
python3 -m venv .venv
source .venv/bin/activate
```

Verify: `which python` should point to `.venv/bin/python`.

INSTALL DEPENDENCIES AND PIN THEM:

```
pip install scikit-learn joblib python-dotenv
pip freeze > requirements.txt
```

Open `requirements.txt` and inspect: note the pinned versions and the transitive dependencies that were pulled in automatically. Count how many lines `pip freeze` produced compared to the three packages you installed directly.

THE HABIT TO BUILD: after installing any package, immediately run `pip freeze > requirements.txt` and commit the change. Stale requirements files are the number one cause of "works on my machine" incidents.

Exercise: Configuration and Secrets

WRITE `.env.example`. Create the file with two entries:

```
# API key (secret, never a real value here)
SECRET_API_KEY=replace-me

# Debug mode (never true in production)
DEBUG=true
```

Then create your actual `.env` by copying the example and replacing the placeholder with a real value. Confirm that `.env` does not show up in `git status` because it is already in `.gitignore` from Block 2.

UPDATE `.gitignore`. Add the entries that now matter:

```
.venv/
*.pkl
data/
.env
```

Run `git status` before and after to see the difference.

Exercise: Scaffolding and Security Audit

SCAFFOLD THE PROJECT DIRECTORIES. Create only what we need right now; other folders appear in the block that first needs them:

```
mkdir -p app data models
touch app/__init__.py
```

RUN `pip audit`:

```
pip install pip-audit
pip-audit
```

Inspect and discuss any findings. What is a CVE? What would you do if a critical vulnerability is found in one of your dependencies?

COMMIT THE SCAFFOLD:

```
git add app/ requirements.txt .env.example .gitignore
git commit -m "Add project scaffold, deps, and env template"
git push
```

Exercise: Replicate on the Server

SSH INTO THE SERVER, pull the latest code, and recreate the environment:

```
cd /opt/pixelwise
git pull
bash setup-server.sh
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

The reproducibility promise in action: the setup script provisions system packages, the virtual environment isolates Python, and `requirements.txt` pins every dependency. A fresh VM goes from empty to running in these commands.

TAKE A SNAPSHOT of both VMs. Name them `B3-complete`.

Security Thread

`pip audit` catches known CVEs in dependencies; run it regularly and before every deploy. `.env.example` makes the secrets contract explicit without exposing values. Never commit `.env`. The `.venv/` folder must be in `.gitignore`: it is large, machine-specific, and reconstructable from `requirements.txt` alone.

Self-Reflection and Recap

Reflection questions:

- Why do we use a virtual environment instead of installing packages system-wide?
- What is the difference between `.env` and `.env.example`? Why do we need both?
- What does `pip freeze` capture that you did not explicitly install?
- If a colleague clones your repo, what commands do they run to get a working environment?
- What is the risk of committing `.env` to Git, even briefly?

Recap:

- `setup-server.sh` makes system-level provisioning reproducible and auditable.
- Virtual environments isolate Python packages per project; never install globally.
- `requirements.txt` pins every dependency so both machines run identical code.
- `.env.example` documents secrets; `.env` holds the real values and never enters Git.
- `pip audit` catches known vulnerabilities in your dependency tree.

Milestone: PixelWise has `app/`, `data/`, `models/`, a virtual environment, pinned dependencies, a setup script, and a minimal `.env.example`. The `.gitignore` is doing real work. Anyone who clones the repo can recreate the environment in a handful of commands.

Teaser. The repository now has a scaffold, pinned dependencies, and a provisioned server. But there is no application code yet. In the next block we integrate a trained machine learning model, write inference logic, and make PixelWise actually classify a digit for the first time.

Index

.env, 39
.env.example, 39
.gitignore, 29
/etc, 19
/opt, 19
12-factor app, 39

Ansible, 37
assessment, 9

Bash, 14

chmod, 14
chown, 14
conda, 38
continuous integration, 28
course work, 9
CVE, 39

data hygiene, 25
dependencies, 34
dev VM, 11
distribution, 14
Docker, 40
Dockerfile, 40
DVC, 25

Ed25519, 15
exercises, 17, 19, 20, 30–33, 42–44

fork, 28

Git, 21
  -u flag, 28
  blob, 25
  branch, 25
  branching, 26
  clone, 28
  commit, 25
  data model, 25
  default branch, 32
  fetch, 28
  fork, 28
  HEAD, 25
  history, 29
  HTTPS clone, 33
  install, 30
  LFS, 25
  merge conflict, 27
  merging, 26
  pull, 28
  push, 28, 33
  remotes, 27
  repository, 26
  staging, 26
  staging area, 26
  status, 32
  tag, 42
  tags, 27
  tree, 25
  working directory, 26
GitHub, 21, 27
  username, 33

Hugging Face, 25
hypervisor, 13

interface contracts, 24
issues, 28

KeePass, 15

least privilege, 14
license, 2
Linux, 9, 14
lock file, 40

MNIST, 11

nano, 16, 32

OpenSSH, 15
Oracle VirtualBox Manager, 9

pip, 34, 38
pip audit, 39, 45
PixelWise, 11
poetry, 40
project report, 9
project structure, 34
Projektbericht, 9
pull request, 28
PuTTY, 15
PyPI, 38
pyproject.toml, 40
python-dotenv, 39

recap, 21
recovery, 42
reflection, 21
Remote-SSH, 15
requirements.txt, 38

secrets, 29
semantic versioning, 27
server, 11
setup-server.sh, 35, 37
snapshot, 13
SSH, 9, 15
  config, 30
  key, 30
  keygen, 30
  login, 33
  private key, 30
  public key, 30
ssh-agent, 15
sudo, 30
systemctl, 16

systemd, 16

tig, 27
touch typing, 19
transitive dependencies, 40
type-1 hypervisor, 13
type-2 hypervisor, 13

Ubuntu, 14
  terminal, 30
Unix philosophy, 14
uv, 40

venv, 37
version control, 21, 23

Virtual Machines, 9, 13
virtualenv, 34, 37
VS Code, 15

Weights & Biases, 25
whoami, 14